1 回顾历史

事实上，在 Spring3.0 开始，Spring 官方就已经开始推荐使用 java 配置来代替传统的 xml 配置了，我们不妨来回顾一下 Spring 的历史：

1.1 Spring1.x 时代

在此时因为 jdk1.5 刚刚出来，注解开发并未盛行，因此一切 Spring 配置都是 xml 格式，想象一下所有的 bean 都用 xml 配置，细思极恐啊，心疼那个时候的程序员 2 秒

```
<bean id="helloService" class="com.gpedu.example.service.HelloService"
/>

<bean id="bookService" class="cn.moondev.service.BookServiceImpl">
    <property name="dependencyService" ref="dependencyService"/>
</bean>
<bean id="otherService" class="cn.moondev.service.OtherServiceImpl">
    <property name="dependencyService" ref="dependencyService"/>
</bean>
<bean id="dependencyService" class="DependencyServiceImpl"/>
```

代码在 demo01 中

1.2 Spring2.x 时代

Spring 引入了注解开发，但是因为并不完善，因此并未完全替代 xml，此时的程序员往往是把 xml 与注解进行结合，貌似我们之前都是这种方式。

- @Required/@Repository/@Aspect

spring 2.5

- @Component
- @Service
- @Controller
- @RequestMapping

代码在 demo02 中

```
<context:component-scan base-package="com.gpedu.demo02"/>
```

咕泡出品 必属精品

1.3 Spring3.x 时代

3.0 以后 Spring 的注解已经非常完善了，因此 Spring 推荐大家使用完全的 java 配置来代替以前的 xml，不过似乎在国内并未推广盛行。然后当 SpringBoot 来临，人们才慢慢认识到 java 配置的优雅。

- @Configuration 去 XML 化

代码中 demo03

- @Import :当我们在 web.xml 里面配置时，表示指定配置文件，然后回去配置中读取信息，如果我们要导入的别的配置文件，就可以通过 import 这个配置去做。这个 import 可以写在 applicationContext.xml 中，意味着把多个配置文件合并。

代码在 demo04 中

- @Enable 模块驱动

```
@EnableAspectJAutoProxy org.springframework.c…
@EnableLoadTimeWeaving org.springframework.co…
@EnableMBeanExport org.springframework.contex…
@EnableAsync org.springframework.scheduling.a…
@EnableCaching org.springframework.cache.anno…
@EnableScheduling org.springframework.schedul…
@EnableWebMvc org.springframework.web.servlet…
Press Ctrl+空格 to see non-imported classes  Next Tip
```

在 Spring3.x 版本中，集成 Redis 或者 mybatis 要怎么做：

```
1.导包
2.创建一个配置类
3.通过@bean 注解来声明一个 bean
```

mybatis 配置举例：

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context-4.
3.xsd
        http://www.springframework.org/schema/aop
        http://www.springframework.org/schema/aop/spring-aop-4.3.xsd
        http://www.springframework.org/schema/tx
        http://www.springframework.org/schema/tx/spring-tx-4.3.xsd">

    <!--1 引入属性文件，在配置中占位使用 -->
    <context:property-placeholder location="classpath*:db.properties" />

    <!--2 配置C3P0 数据源 -->
    <bean id="datasource" class="com.mchange.v2.c3p0.ComboPooledDataSou
rce" destroy-method="close">
        <!--驱动类名 -->
        <property name="driverClass" value="${jdbc.driver}" />
        <!-- url -->
        <property name="jdbcUrl" value="${jdbc.url}" />
        <!-- 用户名 -->
        <property name="user" value="${jdbc.uid}" />
        <!-- 密码 -->
        <property name="password" value="${jdbc.pwd}" />
        <!-- 当连接池中的连接耗尽的时候 c3p0 一次同时获取的连接数  -->
        <property name="acquireIncrement" value="5"></property>
        <!-- 初始连接池大小 -->
        <property name="initialPoolSize" value="10"></property>
        <!-- 连接池中连接最小个数 -->
        <property name="minPoolSize" value="5"></property>
        <!-- 连接池中连接最大个数 -->
        <property name="maxPoolSize" value="20"></property>
    </bean>

    <!--3 会话工厂 bean sqlSessionFactoryBean -->
    <bean id="sqlSessionFactory" class="org.mybatis.spring.SqlSessionFa
ctoryBean">
        <!-- 数据源 -->
        <property name="dataSource" ref="datasource"></property>
        <!-- 别名 -->
        <property name="typeAliasesPackage" value="com.zhangguo.booksto
re.entities"></property>
        <!-- sql 映射文件路径 -->
        <property name="mapperLocations" value="classpath*:com/zhangguo
/bookstore/mapper/*Mapper.xml"></property>
    </bean>

    <!--4 自动扫描对象关系映射 -->
    <bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
```

咕泡出品 必属精品

```
        <!--指定会话工厂，如果当前上下文中只定义了一个则该属性可省去 -->
        <property name="sqlSessionFactoryBeanName" value="sqlSessionFac
tory"></property>
        <!-- 指定要自动扫描接口的基础包，实现接口 -->
        <property name="basePackage" value="com.zhangguo.bookstore.mapp
er"></property>
    </bean>

    <!--5 声明式事务管理 -->
    <!--定义事物管理器，由spring 管理事务 -->
    <bean id="transactionManager" class="org.springframework.jdbc.datas
ource.DataSourceTransactionManager">
        <property name="dataSource" ref="datasource"></property>
    </bean>
    <!--支持注解驱动的事务管理，指定事务管理器 -->
    <tx:annotation-driven transaction-manager="transactionManager"/>

    <!--6 容器自动扫描IOC 组件  -->
    <context:component-scan base-package="com.zhangguo.bookstore"></con
text:component-scan>

    <!--7 aspectj 支持自动代理实现AOP 功能 -->
    <aop:aspectj-autoproxy proxy-target-class="true"></aop:aspectj-auto
proxy>
</beans>
```

通过上面这种方式才能把 mybatis 注入到我们 springIOC 容器中，但是这种方式太麻烦了

怎么做更简便呢？

@Enable 启动一个模块，把相关组件的 bean 自动装配到 IOC 容器中

没有使用 Enable 启动一个模块时的案例看 demo05

使用 Enable 启动一个模块时的案例看 demo06

1.4 Spring4.x 时代

- @Conditional

代码在 demo07 中

springboot 条件注解

@ConditionalOnBean（仅仅在当前上下文中存在某个对象时，才会实例化一个 Bean）
@ConditionalOnClass（某个 class 位于类路径上，才会实例化一个 Bean）
@ConditionalOnExpression（当表达式为 true 的时候，才会实例化一个 Bean）
@ConditionalOnMissingBean（仅仅在当前上下文中不存在某个对象时，才会实例

路漫出品 必属精品

化一个 Bean）
@ConditionalOnMissingClass（某个 class 类路径上不存在的时候，才会实例化一个 Bean）
@ConditionalOnNotWebApplication（不是 web 应用）

1.5 IOC 流程

1.我们要把托管给 spring 的 bean 先扫描到 beanDefinition，并初始化 Bean 容器，放到 BeanDefinitionMap 容器！

2.然后实例化，再进行属性注入（DI)

3.初始化 Bean,包括是否实现了 BeanNameAware 等接口

4.是否有实现了 BeanPostProcessor 类， 调用 postProcessBeforeInitialization

5.调用 Bean 的初始化方法

6.是否有实现了 BeanPostProcessor 类， 调用 postProcessAfterInitialization，然后这个方法里调用后续 aop 的逻辑

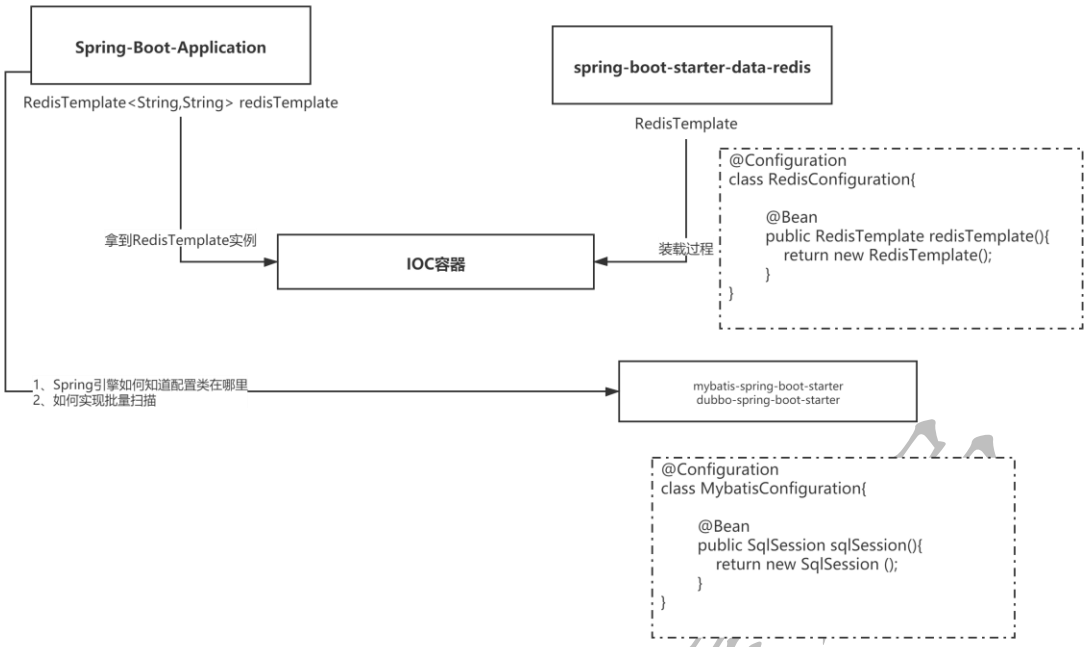
2 springboot 启动流程

见补充讲义内容

3 Springboot 自动装配

代码举例见 spring-boot-demo 中的 redis 使用

咕泡出品 必属精品



3.1 Spring 的动态 Bean 装载

- ImportSelector：代码看 demo02、03、04
- Registator

3.2 SpringFactoriesLoader（SPI）

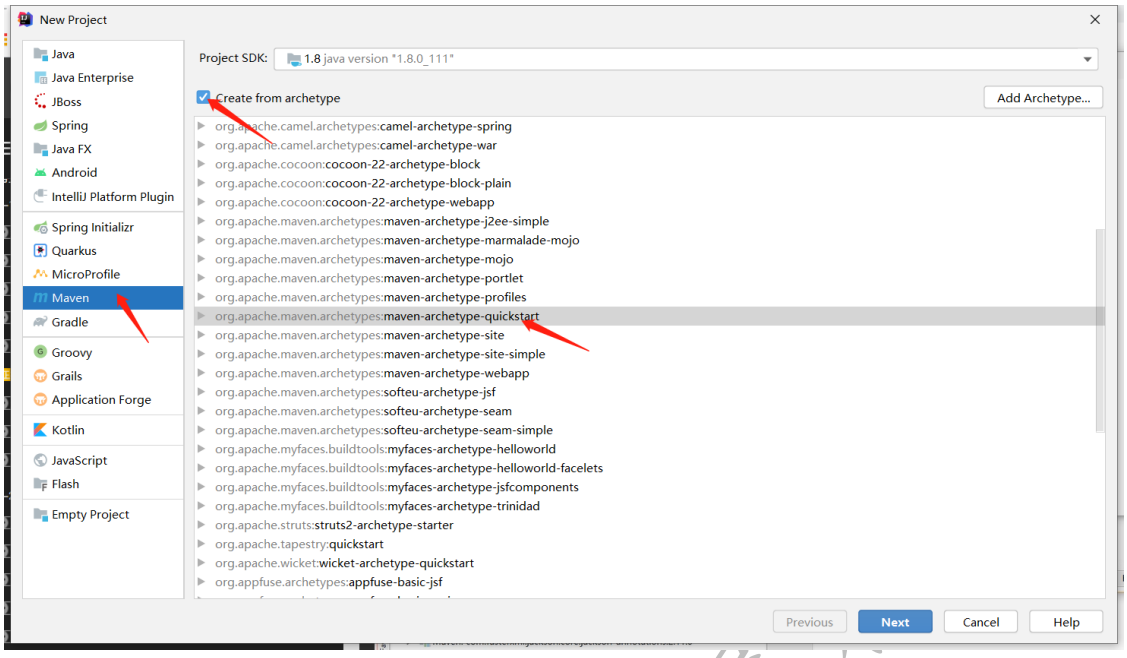
spring.factories（这是 springboot 里面的一种实现），但是我们 JDK 里面也提供了一种实现方式就是 SPI，我们的 Springboot 里面的 spring.factories 都是基于 jdk 中的 SPI 思想做的一种实现。

3.3 SPI 机制

Service provider interface

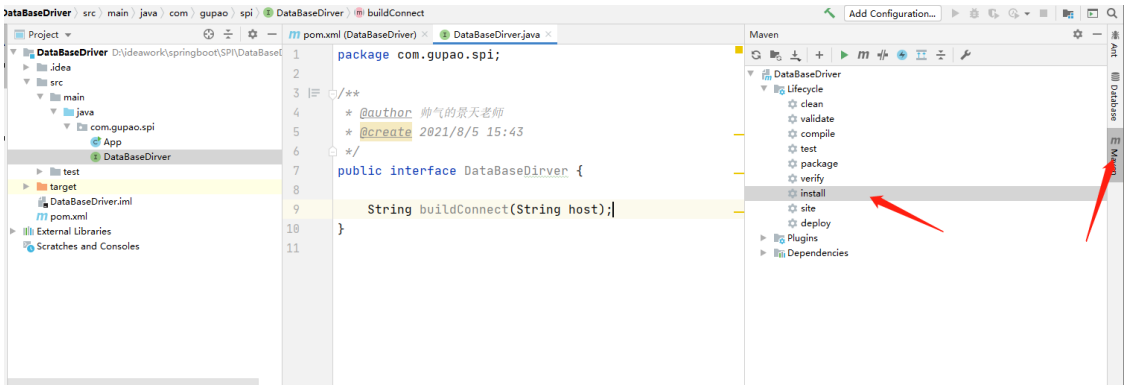
实现方式：

1）创建接口 Maven 项目 DataBaseDriver



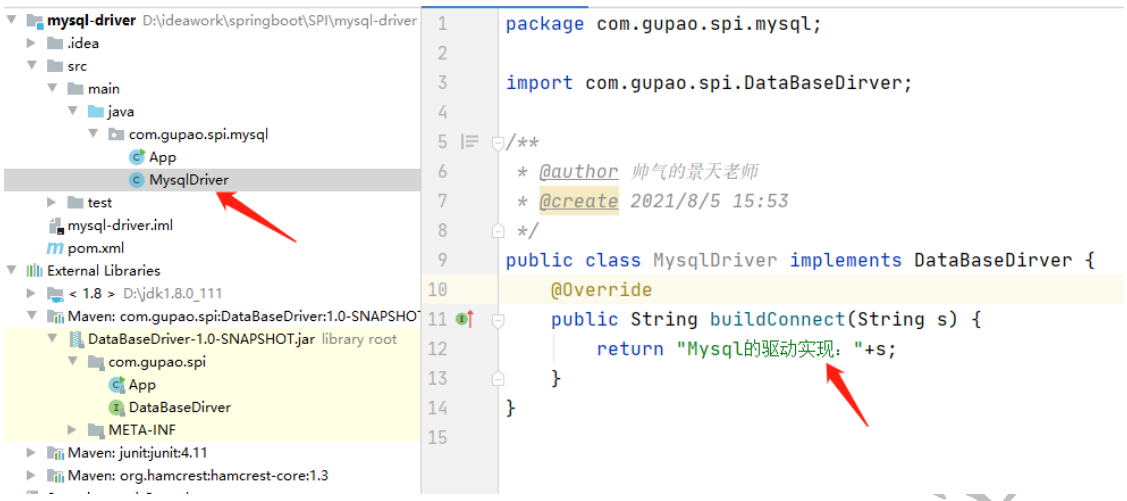
2）然后创建一个接口 DataBaseDriver

3）把接口项目打包成 jar 包

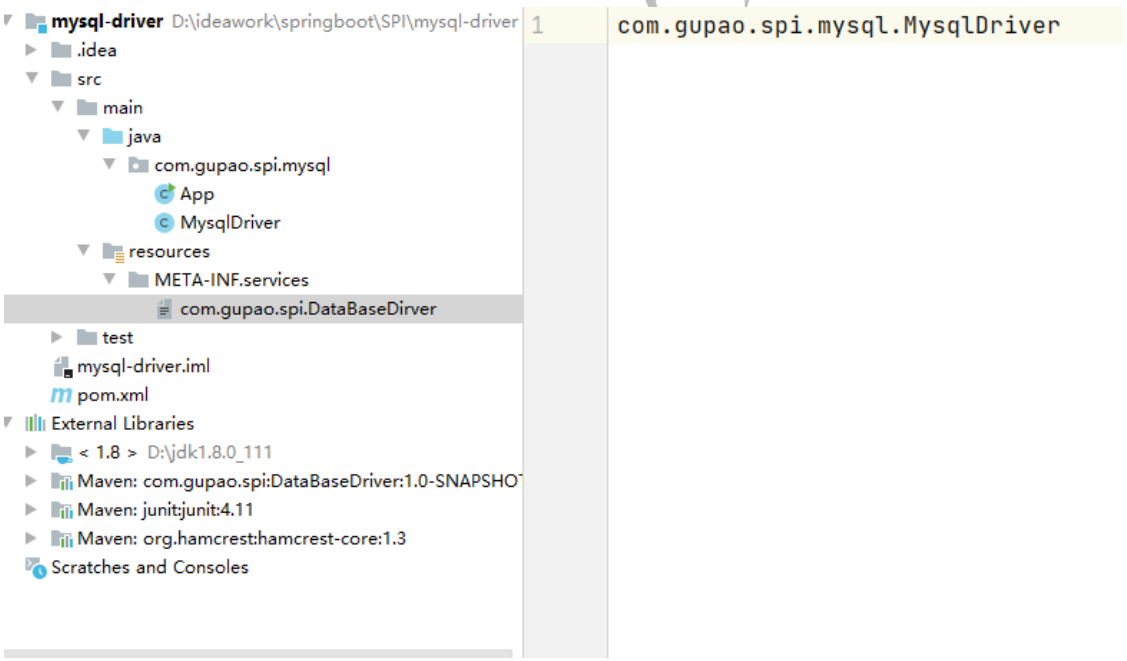


4）创建 mysql-driver 的 Maven 项目，并且引入上面的 jar 包

5）mysql-driver 项目中实现接口



6）开始扩展，首先在 main 目录下创建 resources 目录，然后在 resources 目录下创建 MET-INF 目录，在 META-INF 目录下再创建 services 目录。在 services 目录下创建 com.gupao.spi.DataBaseDriver 文件（这个文件名要跟接口的全路径名一致）。



7）现在就可以开始用它了，我们再创建一个应用程序 app-demo

8）在应用程序中导入接口 jar 包和实现类 jar 包

```
    </dependency>
    <dependency>
      <groupId>com.gupao.spi</groupId>
      <artifactId>DataBaseDriver</artifactId>
      <version>1.0-SNAPSHOT</version>
    </dependency>

    <dependency>
      <groupId>com.gupao.spi.mysql</groupId>
      <artifactId>mysql-driver</artifactId>
      <version>1.0-SNAPSHOT</version>
    </dependency>
  </dependencies>
```

9）写调用方法：

满足以下条件

- 需要在 classpath 目录下创建一个 META-INF/services
- 在该目录下创建一个扩展点的全路径名
  - 文件中填写这个扩展点的实现
  - 文件编码格式 UTF-8
- ServiceLoader 去进行加载

3.4 条件控制

对于官方的 starter 组件，例如 spring-boot-starter-data-redis,他们的 META-INF 目录下不存在 spring.factories

- 官方包 spring-boot-starter-xxx
  - 官方包的 configuration 在 spring-boot-autoconfigure 下，然后会根据条件进行控制，如果引入了包就加载，如果没有引包就不加载

SpringBootDemoApplication.java | spring.factories | RedisAutoConfiguration.class

Decompiled .class file, bytecode version: 52.0 (Java 8) Download Sources Choose S

```java
import ...

@Configuration(
    proxyBeanMethods = false
)
@ConditionalOnClass({RedisOperations.class})
@EnableConfigurationProperties({RedisProperties.class})
@Import({LettuceConnectionConfiguration.class, JedisConnectionConfiguration.c
public class RedisAutoConfiguration {
    public RedisAutoConfiguration() {
    }

    @Bean
    @ConditionalOnMissingBean(
        name = {"redisTemplate"}
    )
    @ConditionalOnSingleCandidate(RedisConnectionFactory.class)
    public RedisTemplate<Object, Object> redisTemplate(RedisConnectionFactory
```

- 第三方包 xxx-spring-boot-starter

4 Starter 组件

手写案例见 starter 目录下的 redisson-spring-boot-starter 和 spring-boot-demo-02 项目

定义：相当于一个模块

读取配置文件：https://blog.csdn.net/qq_37312838/article/details/108237678

5 元注解

- @Target 注解可以用在哪。TYPE 表示类型，如类、接口、枚举
  - @Target(ElementType.TYPE) //接口、类、枚举
  - @Target(ElementType.FIELD) //字段、枚举的常量
  - @Target(ElementType.METHOD) //方法
  - @Target(ElementType.PARAMETER) //方法参数
  - @Target(ElementType.CONSTRUCTOR) //构造函数
  - @Target(ElementType.LOCAL_VARIABLE)//局部变量
  - @Target(ElementType.ANNOTATION_TYPE)//注解
  - @Target(ElementType.PACKAGE) ///包
- @Retention 注解的保留位置。只有 RUNTIME 类型可以在运行时通过反射获取其值
  - @Retention(RetentionPolicy.SOURCE) //注解仅存在于源码中，在 class 字节码文件中不包含
  - @Retention(RetentionPolicy.CLASS) // 默认的保留策略，注解会在 class 字节码文件中存在，但运行时无法获得，
  - @Retention(RetentionPolicy.RUNTIME) // 注解会在 class 字节码文件中存在，在运行时可以通过反射获取到
- @Documented 该注解在生成 javadoc 文档时是否保留
- @Inherited 被注解的元素，是否具有继承性，如子类可以继承父类的注解而不必显式的写下来。

哈泼出品 必属精品